

Conceptos básicos de herencia y sus aplicaciones

Índice

Responsabilidades de clases.....	2
Orden jerárquico entre clases	2
Relación ES-UN	2
Utilidad de la herencia.....	4
Operaciones. Interfaz pública.....	6
Sobreescritura. Alterando el comportamiento heredado.....	6
Uso de la keyword super	7
Sobrecarga de métodos.....	10

Responsabilidades de clases

Hasta aquí vimos anteriormente que para comenzar a resolver un problema utilizando el paradigma de objetos, debemos pasar por el denominado **proceso de abstracción**. Durante este proceso, no solo identificamos las partes que integran nuestro “universo” o “dominio”, sino cómo interactúan esas partes. Es así como llegamos a definir **clases**, con sus atributos y operaciones. Además, definimos qué debe hacer cada parte para contribuir a la solución del problema, es decir, establecimos las **responsabilidades** de las clases.

Orden jerárquico entre clases

Cada vez que intentamos resolver un problema utilizando el paradigma de objetos, debemos modelarlo, definiendo una multitud de clases, cada una con sus atributos y operaciones. Cada clase deberá tener sus responsabilidades bien definidas, pero aun así llegaremos a detectar que algunas son demasiado parecidas entre sí o simplemente tienen algunas pocas similitudes, pero sus marcadas diferencias. Es importante dar a esta multiplicidad de partes algún tipo de orden. Según estas similitudes y diferencias podemos **agrupar** todos aquellos atributos y operaciones en común en un lugar y dejar separadas las diferencias. Es así como nace un orden jerárquico superior e inferior.

Dejaremos en la parte superior a aquellas partes comunes y en el inferior a las diferentes. No se trata de agrupar según mejor o peor, es solo un ordenamiento.

Relación ES-UN

Si decimos que todos los perros tienen un nombre y una edad, y todos ladran y juegan, es así como podemos decir que un perro es un perro porque tiene edad y nombre, y ladra y juega.

Si analizamos un caniche, veremos cómo juega y cómo ladrar; si analizamos un Pug Carlino, también veremos cómo juega y ladra, aunque lo hace muy distinto al caniche. Entonces, tanto el caniche como el Pug ladran y juegan, ambos tienen nombre y edad:

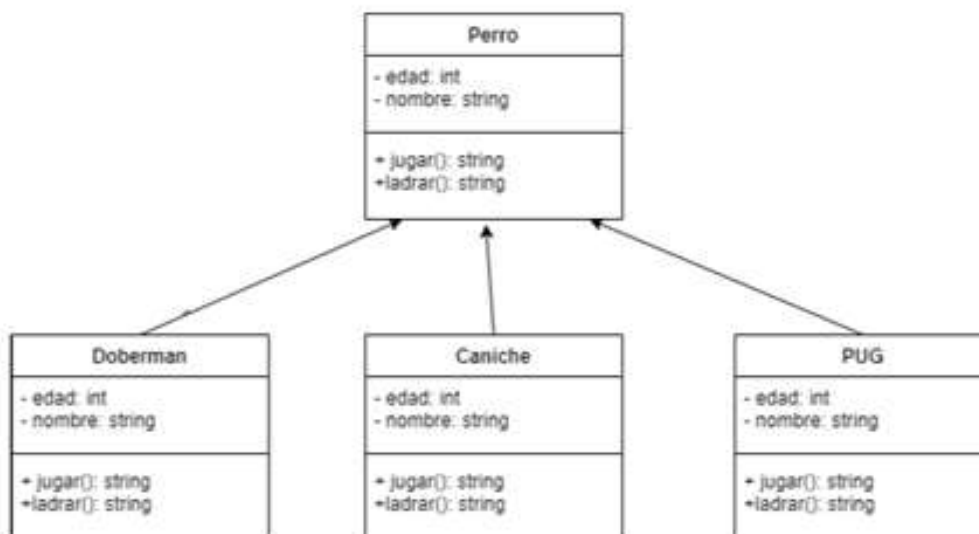
sería razonable asumir que si tienen y hacen todo lo que hace un perro, entonces **es un** perro. El mismo análisis podría hacerse con cualquier otra raza.

En este punto es importante notar que lo que las clases tienen es irrelevante, lo fundamental es lo que las clases hacen. Los atributos de las clases deberían ser siempre privados y tener operaciones para obtener o establecer sus valores. A partir de ahora, entonces, hablaremos siempre del comportamiento de las clases, aquello que hacen → si un perro tiene un nombre entonces no importa el atributo nombre, sino las operaciones para nombrarlo y para obtener su nombre.

El conjunto de las cosas que la clase hace, es decir, el conjunto de operaciones públicas se conoce con el nombre de “interfaz pública” de la clase.

Por todo esto, podemos decir que un caniche ES UN perro. De la misma manera que podríamos decir que profesor ES UN empleado, y más aún: un empleado ES UNA persona, por lo tanto, un profesor ES UNA persona.

Nuevamente, al observar la realidad y pasar por el proceso de abstracción, obtuvimos una serie de entidades que se ordenan naturalmente, y la herencia responde a ello.



Podemos decir entonces que la **herencia** es un **ordenamiento entre clases** que define una **relación “es un”**. Entonces decimos que un Caniche **es un** perro, porque tiene y hace todo lo que hace un perro.

Utilidad de la herencia

Esta es una pregunta interesante, ya que afirmamos que la herencia es uno de los pilares de la orientación a objetos. Si analizamos el esquema anterior, tanto Caniche, como Pug, como Dóberman hacen **lo mismo** que hace el perro. ¿Si hacen lo mismo que el perro, para qué escribir el código de lo que hacen? ¿No sería más conveniente escribirlo una sola vez en la clase Perro y que Dóberman, Caniche, etc. “obtengan” este comportamiento desde Perro?

De hacer esto, decimos que Caniche, Pug, etc. “heredan” el comportamiento de un perro, es decir, la clase Dóberman hereda de la clase Perro todas sus operaciones.

Supongamos la clase Perro:

```
public class Perro {  
  
    private int edad;  
    private String nombre;  
  
    public String ladrar() {  
        return "GUAU";  
    }  
    public String jugar() {  
        return "JUGANDO";  
    }  
    //getters y setters de edad y nombre  
}
```

Y yo quiero crear mi clase Caniche y mostrar que caniche **es un** perro, entonces tenemos que hacer:

```
public class Caniche extends Perro {  
  
}
```

Con la palabra “extends” indico, en código, que un caniche es un perro, y por tanto hereda todo su comportamiento.

Es por eso, que a partir de este punto podemos hacer:

```
Perro p = new Perro();  
System.out.println(p.ladrar()); // < GUAU  
  
Caniche c = new Caniche();  
System.out.println(c.ladrar()); // < GUAU
```

En la clase Caniche no se definió un método “ladrar”. Para que esto compile, y luego funcione, lo que el compilador hace es pensar:

- a. La operación ladrar sin parámetros, ¿está definida en Caniche?
- b. Sí, entonces se compila correctamente y se puede ejecutar luego (no es el caso de este ejemplo).
- c. No, pero intento ir buscarla a la clase de la que extiende Caniche, es decir, en Perro.
 - i. ¿Está la operación (método) ladrar, que no recibe parámetros en Perro?
 - ii. Sí, entonces la sintaxis es correcta.
 - iii. No, intento buscarla en la clase de la que extiende Perro.
 - ➔ ¿Está en esa clase?... Y repetir hasta encontrar el método. Si no se encuentra en ninguna parte de la cadena (de la jerarquía) entonces el código no puede compilarse correctamente.

Al momento de la ejecución, ocurre algo similar, el *runtime* piensa “ladrar sin parámetros, ¿está definida en Caniche? No, pero si está en Perro, voy a ejecutar el código que corresponde a la operación ladrar que se hereda de Perro”.

Es importante que este proceso se repita en cada escalón de la jerarquía, y se va “escalando” hasta encontrar la operación que “matchea” con la cantidad, tipo y orden de parámetros de la llamada.

Acá tenemos la primera consecuencia importante de la herencia. Si vemos la clase Caniche, no repetimos nada del código para hacer que un Caniche ladre, ni juegue, etc. La herencia es un mecanismo que apunta fundamentalmente a la reutilización de código.

Operaciones. Interfaz pública

Todos sabemos que los caniches ladran, también lo hacen los labradores, los siberianos, los dóberman, etc. Con lo que hicimos hasta acá ahorramos código: para cualquiera de estas clases, no deberíamos escribir nuevamente el ladrar, jugar, etc. Lo que sí podríamos hacer es agregar operaciones, entonces, como ya heredamos ladrar y jugar podemos agregarle al Labrador, por ejemplo, una operación cuidarNiños(), o al Dóberman cuidarCasa(). Entonces, el Labrador o el Dóberman **son perros**, pero hacen algo más. Es decir, Labrador **es un perro** porque hace **al menos** lo que hace un perro. También se dice Labrador **es un perro** porque “cumple con la interfaz pública” de perro. Es importante esta distinción, porque la relación **es un** no se da por respetar exactamente lo que hace otra clase, sino que, además, puede agregar comportamiento. Por todo esto es que podemos hacer:

```
Perro p = new Perro();  
p.ladrar();  
  
Perro x = new Doberman(); // Doberman ES UN Perro  
x.ladrar();
```

Sobreescritura. Alterando el comportamiento heredado

Analizando nuevamente al caniche, cuando lo oímos ladrar, claramente ladra de forma muy distinta a un labrador. ¿Cómo se puede mostrar esa diferencia dentro de la **jerarquía** de perros armada hasta aquí? Se utiliza un mecanismo que se llama **sobreescritura de métodos**. Para aplicar sobreescritura, se debe reescribir el método respetando el tipo, orden y cantidad de parámetros suministrados. De esta forma, “pisamos” el comportamiento heredado y lo reemplazamos por otro:

```
public class Caniche extends Perro {
    public String ladrar() {
        return "WIF WIF";// caniche tiene un ladrido mas agudo
    }
}
```

Entonces, si hiciéramos en un main:

```
Caniche c = new Caniche();
System.out.println(c.ladrar()); // < WIF WIF
```

Aquí, la clase Caniche sí define un método “ladrar”. El compilador analiza: “¿ladrar, sin parámetros está definida en Caniche? Sí, entonces la sintaxis es correcta.”

Al momento de la ejecución, nuevamente, el *runtime* analiza: “¿ladrar sin parámetros, está definida en Caniche?

1. Sí: la ejecuto.
2. NO: debo voy a ejecutar el código que corresponde a la operación ladrar que se hereda de Perro”.

Incluso podemos ir más allá, podemos “pisar” el método heredado, pero a la vez podemos reutilizar aquello que heredamos. Doberman suena algo más agresivo, así que cuando ladre, podemos hacer que gruñe un poco también, entonces:

```
public class Doberman extends Perro {
    public String ladrar() {
        return super.ladrar() + " GRRRR"
    }
}
```

Entonces, si hiciéramos un *main*:

```
Doberman d = new Doberman();
System.out.println(d.ladrar()); // < GUAU GRRRR
```

Uso de la keyword super

Vimos aquí que podemos acceder al comportamiento de la clase padre, también llamada superclase, mediante la palabra clave “super”.

De no contar con la palabra `super`, si hiciéramos:

```
public class Doberman extends Perro {
    public String ladrar() {
        return ladrar() + " GRRRR" // loop infinito!
    }
}
```

El método `ladrar()` se llamaría a sí mismo infinitamente. En cambio, con el `super` explicitamos que queremos llamar al `ladrar()` de la superclase.

Cabe aclarar que, si tenemos una jerarquía de 3 o más niveles, **no se pueden** encadenar las llamadas a `super` (es decir, no se puede hacer `super.super.super.unMetodo()`), sino que siempre se hace de a **un** nivel por clase.

Nuevamente, ¿qué ocurrió aquí? **reutilizamos** código. El Dóberman retorna "GUAU GRRRR" cuando ladra, pero el código del ladrido que devuelve "GUAU" lo escribimos solo una vez, en un solo lugar, en la clase `Perro`.

La palabra clave `super` no solo se utiliza en las llamadas a métodos. También se utiliza `super` para hacer llamadas a los constructores de la clase padre, por ejemplo:

```
public class Perro {
    private String nombre;

    public Perro(String nombre) {
        this.nombre = nombre;
    }
    public String ladrar() {
        return super.ladrar() + " GRRRR"
    }
}

public class Doberman extends Perro {
    private int agresividad;

    public Doberman(int agresividad) {
        this.agresividad = agresividad;
    }

    // quiero un constructor que le pase el nombre
    public Doberman(String nombre) {
```



```

        //this.nombre = nombre;< este código ya lo escribí en
Perro!
        super(nombre); //RE-utilizo lo que ya escribi
    }

    public String ladrar() {
        return ladrar() + " GRRRR" // loop infinito!
    }
}

```

Aun así como está, Doberman, arrojaría un error de compilación. El constructor que recibe el nivel de agresividad hace uso de la palabra `this`. Esa línea de código asume que ya existe una instancia de Doberman para darle la agresividad. Doberman **es un** perro, y para el momento de darle la agresividad al Doberman, su “parte Perro” (aquello que hace que un dóberman sea un perro) ya debería existir, para luego asignarle la agresividad. Para ello, se agrega “implícitamente” una llamada a `super()`, sin parámetros (el constructor por default). De esta manera, tengo a la parte de Perro de un Doberman para luego asignarle la agresividad. Como vimos en el apunte de constructores, si se definen constructores adicionales al constructor por default, este se pierde. Entonces, en este caso, es importante declararlo explícitamente.

El ejemplo con el código correcto sería:

```

public class Perro {
    private String nombre;

    public Perro() {

    }

    public Perro(String nombre) {
        this.nombre = nombre;
    }
    public String ladrar() {
        return super.ladrar() + " GRRRR"
    }
}

public class Doberman extends Perro {
    private int agresividad;

    public Doberman(int agresividad) {

```

```

        //aquí la llamada implícita a super() no tiene
        problemas
        this.agresividad = agresividad;
    }

    // quiero un constructor que le pase el nombre
    public Doberman(String nombre) {
        //this.nombre = nombre;< este código ya lo escribí en
        Perro!
        super(nombre); //RE-utilizo lo que ya escribi
    }

    public String ladrar() {
        return ladrar() + " GRRRR" // loop infinito!
    }
}

```

Sobrecarga de métodos

Existe otra forma de reutilizar código, pero en este caso no necesariamente se da dentro de una jerarquía. Se la denomina sobrecarga de métodos. Aquí se trata de utilizar el código de métodos que ya tenemos dentro de nuestra clase o métodos que tenemos en nuestra clase gracias a que los estemos heredando.

Supongamos que tenemos la clase: PerroRobot, que por más que sea una máquina, si hace **al menos** lo que hace un perro (respeto su interfaz pública), entonces **es un** Perro.

En código lo haríamos de la siguiente manera:

```

public class PerroRobot extends Perro {
    public String ladrar() {
        return super.ladrar() + " BIP BIP"; // loop infinito!
    }
}

```

Supongamos ahora que la ventaja de un perro robótico fuera controlar el volumen al que ladra. Entonces podríamos definir una operación que se llame “ladrarConVolumen”. Conceptualmente, un perro ladra, controlar a distancia el volumen del ladrido del perro no es algo que pueda lograrse fácilmente. Los perros ladran y ya. Lo que se puede hacer es definir una nueva operación ladrar, que, conceptualmente e incluso semánticamente,

concuerda con las acciones de un perro, aunque tendrá una variante, un argumento adicional:

```
public class PerroRobot extends Perro {
    public String ladrar() {
        return super.ladrar() + " BIP BIP";
    }
    public String ladrar(int volumen) {
        return this.ladrar() + " VOL:" + volumen;
    }
}
```

Entonces, podríamos hacer:

```
PerroRobot pr = new PerroRobot();
System.out.println(pr.ladrar()); //ladra
//pero está muy fuerte...
System.out.println(pr.ladrar(5)); //ladra y deja dormir...
```

Anteriormente vimos la sobrecarga de constructores, sus restricciones y cómo se decide la ejecución de uno u otro según el tipo, orden y cantidad de parámetros. Eso es solo un caso particular de la sobrecarga en general, que es la que define esas reglas, por lo tanto, en la sobrecarga de métodos aplican las mismas reglas: para sobrecargar un método se debe alterar tipo orden y cantidad de parámetros.

En la sobreescritura tengo que respetar tipo, orden y cantidad de parámetros para poder lograr “pisar” el método heredado. De lo contrario, estaría haciendo una simple sobrecarga entre el método que estoy escribiendo y el que estoy heredando. Por ejemplo:

```
public class PerroRobotV2 extends PerroRobot {

    public String ladrar(int volumen) {
        return this.ladrar() + " DECIBELES:" + volumen; //estoy pisando
    }

    public String ladrar(long volumen) {
        //aca no piso el ladrar porque el tipo del parámetro es otro
        //“caigo” es una sobrecarga
        return "this.ladrar()" + " MAX:" + (volumen*100);
    }
}
```